



DATA STRUCTURES

CONTENTS



4. Linked List Implementation of List ADT

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. This is the fourth module of the course and we will be talking about the linked list implementation of List ADT.

Learning Objectives

The learning objectives of this module are as follows:

- To understand Linked List Implementation of List ADT
- To explore the singly linked list, the circular list and the doubly linked list implementations of List ADT
- To know about the representation of a polynomial as a linked list

4.1 Linked List Implementation (Pointer)

Sometimes it is not possible or realistic to ensure that the list is stored contiguously as it may not be possible to determine the size of the list required beforehand. We may end up wasting space because we have overestimated the requirements. In such cases we use a linked list. In this case we have a series of structures or locations that are not necessarily adjacent in memory. Compared to the array implementation, the pointer implementation uses only as much space as is needed for the elements currently on the list but however requires additional space for the pointers in each cell of the linked list.

Linked lists require some additional steps of code during implementation, but however have some specific advantages as the linked list is dynamic as they can easily grow and shrink in size. We don't need to know how many nodes will be in the list. The locations necessary for the linked list are obtained as when needed.

4.2 Easy and Fast Insertions and Deletions

To insert or delete an element in an array what is needed is to copy to temporary variables to make room for new elements and close the gap caused by deleted elements. With a linked list, there is no need to move other nodes. We need to only reset some pointers. In other words for a linked list implementation there is no need to estimate the maximum size of the list and consequently there is no wasted space. The time taken for some of the common operations such as `printList`, `find` and `findKth` is of the order of $O(n)$ while insert and delete is of the order of $O(1)$. Insert at position 0 (making a new element) or inserting does not require moving the other elements.

Similarly delete at position 0 or deleting requires no shifting of elements. Insertion and deletion becomes easier, but finding the K th element for the place where insertion or deletion needs to be carried out requires time of the order of $O(n)$.

4.3 Linked Lists

A linked list is essentially a series of nodes connected by links where each node contains some data (any type) and a pointer to the next node in the list. In some cases there may be a **Head** which is a pointer to the first node. The Head node has null data value with its pointer pointing to the first node in the list. For convenience we do not show the contents of the head node except the pointer to the first node in the

list. The last node of a linked list normally points to NULL (Figure 4.1). Note, that the nodes in a linked list can be **spread out** over the memory and need not occupy contiguous memory locations. In all the discussions regarding insertion and deletion operations on lists we assume that the list is not empty and hence we do not discuss boundary conditions.

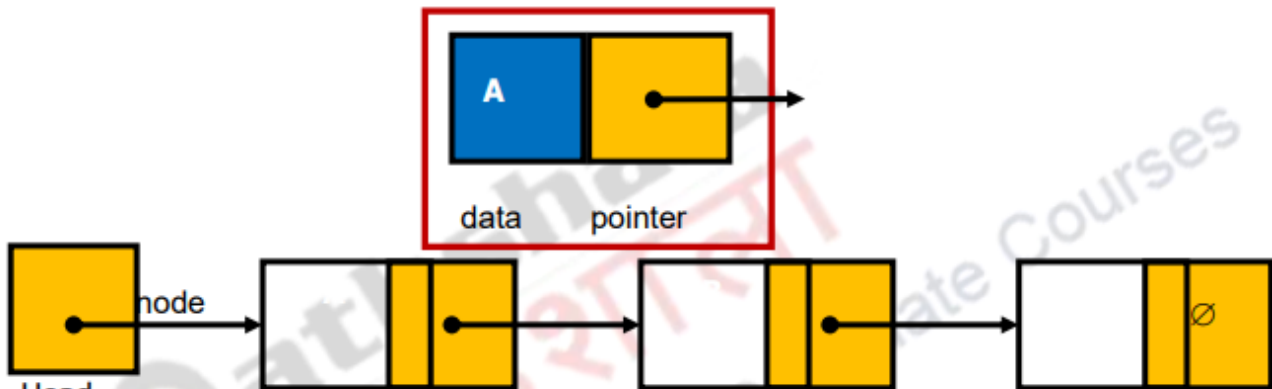


Figure 4.1 Linked Implementation of List ADT

4.3 Insertion- The Scenario

Let us study the insertion scenario in the case of linked lists. You have a linked list which may be empty or not, may be ordered, or not. You want to add an element into the given linked list (Figure 4.2).

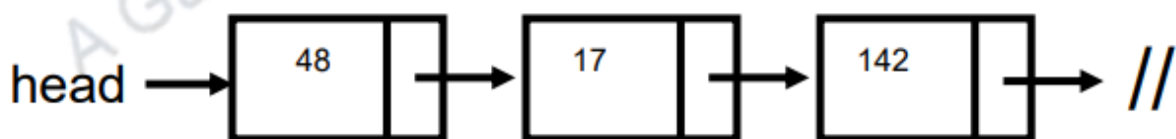


Figure 4.2 Example of a Linked List

When inserting a new node into a linked list, there are four possible cases as described below:

1. Insert into an empty list
2. Insert in front
3. Insert at back
4. Insert in middle

However in fact, we need to handle only two cases. The insert as the first node can handle Case 1 and Case 2. Insert in the middle or at the end of the list Case 3 and Case 4 can be handled in a similar manner.

4.3.1 The Actual Insertion

The actual insertion into a linked list involves the following steps:

- Creation of a new node – We use a function Freenode that gives a node in the structure required to be used by the linked list .
- Adding the data – We add the data to the data component of the node provided by Freenode.
- Updating the next pointer of the new node – We update the Next pointer (pointer pointing to next node in the linked list) of the new node just obtained to point to the appropriate node.
- Update current to point to new node – Now we update the current node's (obtained through appropriate search procedure) next pointer to point to the new node.

4.3.2 Inserting to the Empty Or to the Front of a Linked List

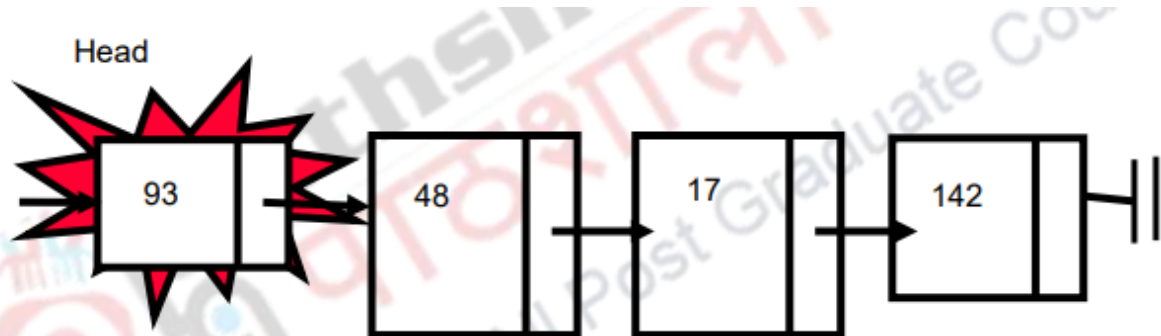


Figure 4.3 Inserting to the Empty Or to the Front of a Linked List

Let us assume that we call inserting into the front of the list as follows:

Insertfront (93, List)

In this case there is no work to find the correct location. Empty or not, head will point to the right location. For the list shown in Figure 4.3, the following are the steps (Figure 4.4)

Steps for Inserting in front

1. **FreeNode(NewNode)** – Get a free node in which to insert the data
2. **NewNode.Data $\Rightarrow$ 93** – Set new node's data component to 93
- 3 **NewNode.Next $\Rightarrow$ Head** – since we are inserting to the Front this new node's next node is the current Head or in case it is an originally empty list than new node's next node is null.
- 4 **Head $\Rightarrow$ NewNode** – since we are inserting to the Front, this node is the new Head

Figure 4.4 Steps for Inserting in Front of a Linked List

4.3.3 Inserting to the End of a List

Let us assume that we call inserting to the end of the list as follows:

Insertend (83, List)

In this case there is a need to find the correct location for the insertion to take place. We need to know the address of the current last node in order to connect to the new last node. We need to perform iteration (Figure 4.5) in order to find the address of current last node for which we need a search pointer (P).

Steps –Iteratively Finding Node

- Initially P $\Rightarrow$ Head**– we start the search from the first node that is pointed to by Head.
- Search until P.Next = Null**- Go on iteratively changing P till we find a node whose next points to Null – indicating that this is the last node.
- Now **P** is the last node after which we are to insert a node.

Figure 4.5 Steps for Iteratively Finding a Node

Now we need to insert a node after the Node P which we have found through the

iterative procedure. The steps are shown in Figure 4.6.

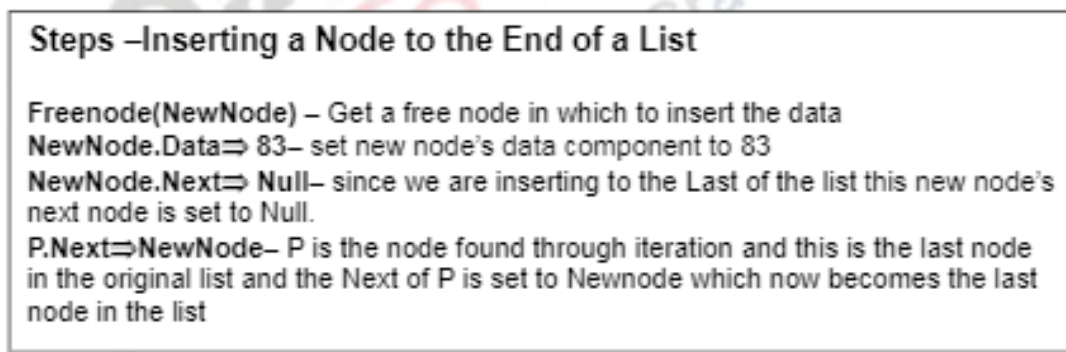


Figure 4.6 Steps for Inserting to the End of a Linked List

4.3.4 Inserting into the Middle of Unsorted List & Sorted List

In this type of insertion also we need to traverse the list recursively until we find the correct place to insert either after a particular given data value or insert at the correct place in a sorted list.

4.3.4.1 Inserting in the Middle

The case we will first consider is inserting in the middle of an unsorted list is where we insert in the middle of the list because we have found the node containing a data value (17) before which we want to insert the given data (88).

Insertbefore (88,17,List)

As discussed previously, the operation insert takes us to the iterative procedure for finding the position after which we want to insert or the data value before which we want to insert. Note here that we need the address of the node before the node containing 17 in order to carry out the insertion. This finding of appropriate position is a common need (Figure 4.7). After finding the position P, the node after which we want to insert, we carry out the insertion (Figure 4.8 and Figure 4.9)

Steps –Iteratively Finding Node

Initially $P \Rightarrow \text{Head}$ – we start the search from the first node that is pointed to by Head.

Search until $P.\text{Next}.\text{Data} = 17$ - Go on iteratively changing P till we find a node whose next points to a node that contains the data item 17. Note here that we need the address of the node before the node containing 17 in order to carry out the insertion

Now P is the node after which we are to insert a node.

Figure 4.7 Steps for Iteratively Finding a Data Item

Steps –Inserting a Node in the Middle of a List

Freemove(NewNode) – Getting a free node in which to insert the data

1. **NewNode.Data $\Rightarrow$ 88**– set new node's data component to 88

2. **NewNode.Next $\Rightarrow$ P.Next**– since we are inserting in the middle of the list, this new node's next should point to the node containing 17

3. **P.Next $\Rightarrow$ NewNode**– P is the node found through iteration and this is the node before node containing 17 and the Next of P is set to Newnode which now becomes a node (with data 88) in the list

Figure 4.8 Steps for Inserting In the Middle of a Linked List

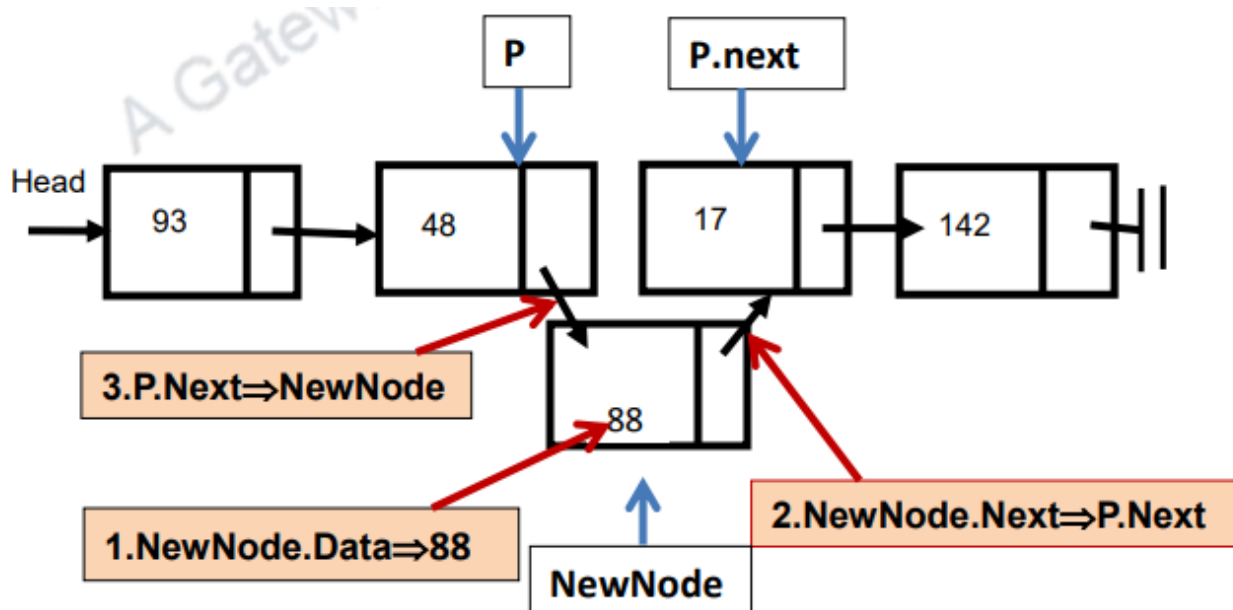


Figure 4.9 Inserting to the Middle of a Linked List

4.3.4.2 Inserting Into a Sorted List



Figure 4.10 Example of a Sorted Linked List

The next case of inserting in the middle of a list is insert a data item into a given sorted list (Figure 4.10) by using the following:

Insertsorted(50,List)

For this we need to find the node after which we want to insert the new data so that the sorted order is maintained. Again a iterative procedure where we need to start from Head using a search variable P until we find the address of the node after which we need to insert – for this we need to ensure that the node after the one we find has value greater than 50 (Figure 4.11). Here we assume the list is in ascending order.

Steps –Iteratively Finding Node After which to Insert in Sorted List

Initially P $\Rightarrow$ Head– we start the search from the first node that is pointed to by Head.

Check if P.Data > 50 then here the case is the same as inserting in the front of a list.

Loop until P.Next.Data > 50– Go on iteratively changing P till we find a node whose next points to a node that contains data item that is greater than 50. Note here that we need the address of the node before the node containing 50 in order to carry out the insertion in sorted order

Now P is the node after which we are to insert a node.

Figure 4.11 Steps for Iteratively Finding a Location to Insert in a Sorted List

Once we have found the location P after which to insert by using the steps indicated

in Figure 4.11 then the procedure for insertion is similar to insertion in the middle of an unsorted list (Figure 4.9)

4.4 Deletion-The Scenario

For deletion, we begin with an existing linked list that could be empty or not and could be ordered or not. In the case of deletion there are three possible situations:

- Delete the first element
- Delete the first occurrence of an element
- Delete all occurrences of a particular element

4.5 The Actual Deletion

Deletion involves the following basic steps:

- Getting to the correct position – iteratively finding the correct node to delete
- Moving a pointer so nothing points to the element to be deleted (Figure 4.12 and Figure 4.13)

We want to **Delete(40, Head)**

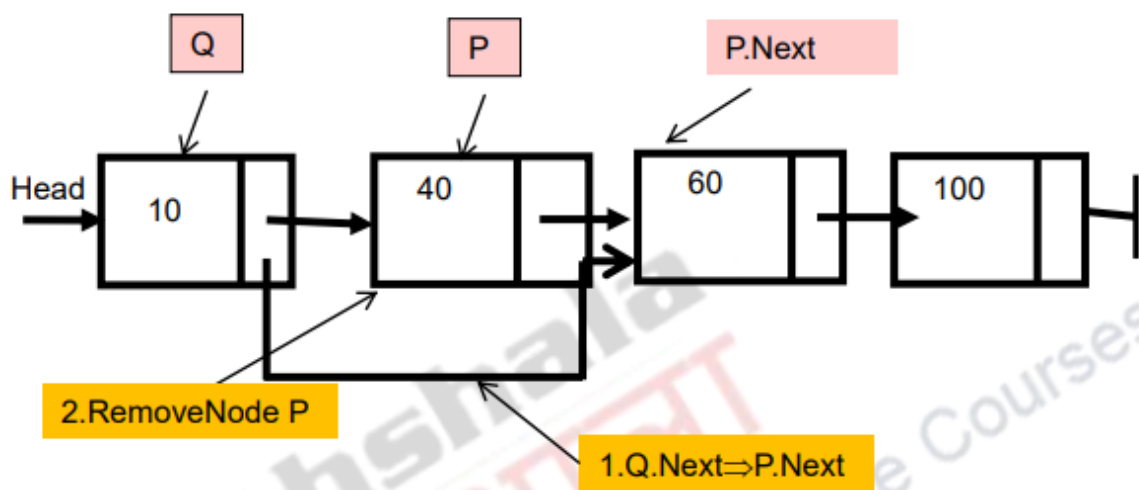


Figure 4.12 Deleting From a Doubly Linked List

Steps –Deleting a Node from a List

Q is the node after which we want to delete a node and **P** is the node we want to delete.

1.Q.Next $\Rightarrow$ P.Next– set the next of the node (Q) previous to the node we want to delete (P) to node to be deleted's next. In this way P is no longer a part of the list.

2.RemoveNode P– Now P is removed from the list.

Figure 4.13 Deleting a Node from a List

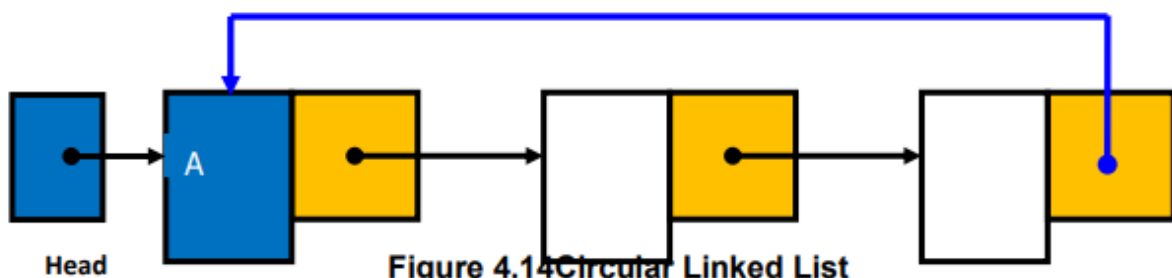
4.6 Linked List Traversal & Printing

In general, traversal means “visiting” or examining each node. In the case of a singly linked list we generally start at the beginning of the list and go one node at a time until the end. This is a recursive procedure (function) that given a head pointer, looks at just one node at a time.

4.7 Circular Linked Lists

Another representation of the list is a variation of the singly linked list called the circular linked list (Figure 4.14) where the last node points to the first node of the list. One question associated with circular linked lists is how do we know when we have finished traversing the list? In order to solve this problem we need to check if the pointer of the current node is equal to the head. We may need to cycle through a list repeatedly.g.

when we use the list to implement a round robin system for a shared resource. In this case having the last node point to the first node is a solution.



4.8 Doubly Linked Lists

Another important representation of lists is the **doubly linked list** which has **two** references in each node; one to the **next** element in the list and one to the **previous** element. This makes moving back and forth in a list easier, and eliminates the need for a previous reference in particular algorithms. The only disadvantage is that it requires a bit more overhead when managing the list.

A doubly linked list provides a natural implementation of the List ADT. Nodes store data, link to the previous node and link to the next node (Figure 4.15).

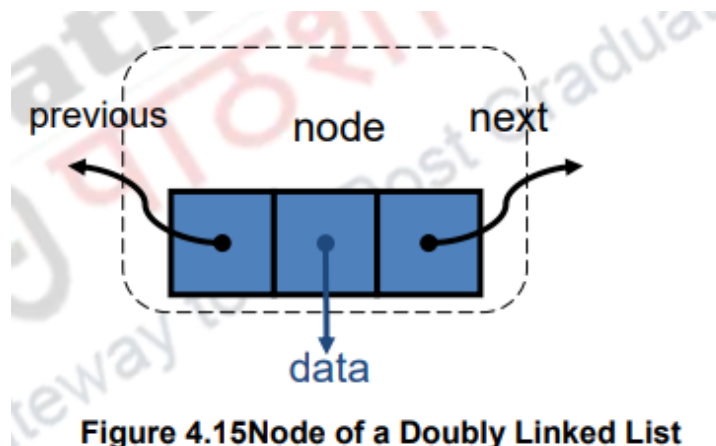


Figure 4.15 Node of a Doubly Linked List

Figure 4.15 Node of a Doubly Linked List

4.8.1 Sentinel Nodes

In order to simplify implementation and to avoid the empty list to be treated as special cases, two specific nodes have been added at both ends of the doubly-linked list. These are the head and tail which are dummy nodes. These nodes are also called sentinels. They do not store any data elements. The Head or the header sentinel has a **null-prev** reference (link) and the Tail or the trailer sentinel has a **null-next** reference (link).

A doubly-linked list in this case can be accessed using the following:

- Reference to sentinel **head**-node;
- Reference to sentinel **tail**-node; and possibly
- A **Size**-counter that keeps track of the number of nodes in the list (excluding the two sentinels).

4.8.2 Empty Doubly-Linked List

Using sentinels, we have no null-links in the actual list nodes. The empty list is as shown in Figure 4.16 where the two sentinels point to each other.

```
head.prev=>null  
head.next=> tail  
tail.prev=> head  
tail.next=>null
```

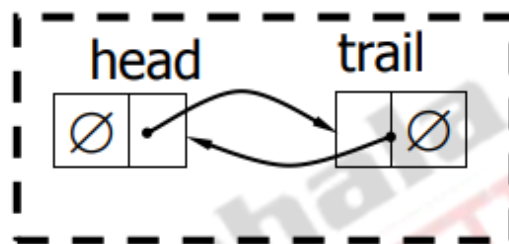


Figure 4.16 An Empty Doubly Linked List with Sentinels

4.8.3 Single Node List

Now let us see how a doubly linked list with a Single Node will look like. This single node is the first node, and is also the last node. The first node is `head.next` and is also the `tail.prev` (Figure 4.17).

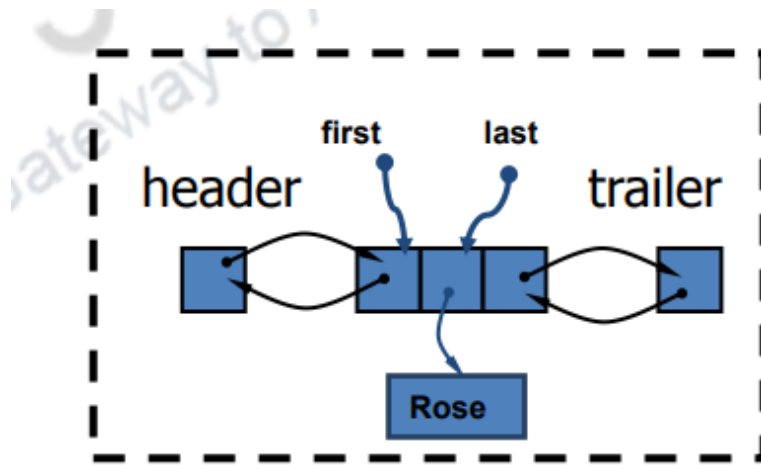


Figure 4.17A Doubly Linked List with a Single Node

4.8.3 Advantages over Singly-linked Lists

The Doubly Linked Lists has a number of advantages over singly linked lists. One important advantage is the quick update operations such as insertions, deletions at both ends (head and tail), and also in the middle of the list.

4.8.4 Insertion into Doubly Linked List (DLL)

The possible cases of inserting a node into a doubly linked list include insertion:

1. At the front of the Doubly Linked List
2. After a given node
3. At the end of the Doubly Linked List

4.8.4.1 Insertion at the front of Doubly Linked List

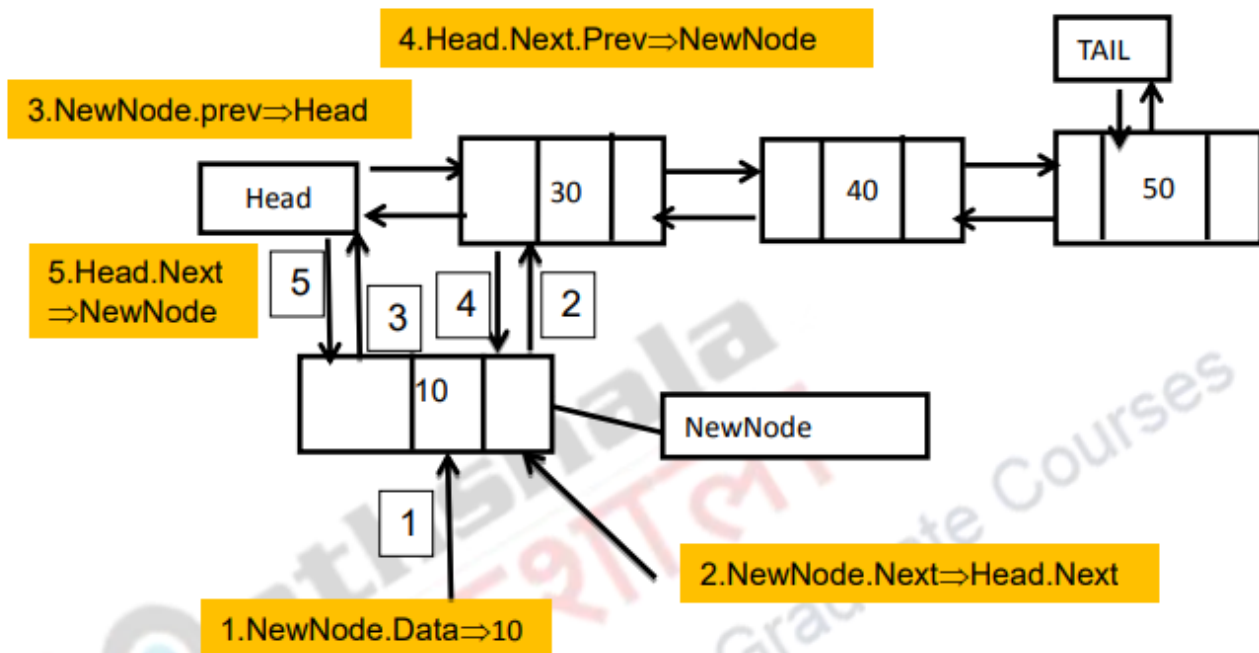


Figure 4.18 Insertion at the Front of a Doubly Linked List

The insertion at the front of a doubly linked list with sentinels is shown in Figure 4.18 and the steps are described in Figure 4.19. Let us assume we call the function:

Insertfront(10,List)

Steps –Inserting a Node at the Front of a Doubly Linked List

Freenode(NewNode) – Getting a free node in which to insert the data. The node consists of three components **Prev**(pointing to previous Node in the list), **Data** (containing data) and **Next** (pointing to next Node in the list)

1.NewNode.Data $\Rightarrow$ 10– set new node's data component to 10

2.NewNode.Next $\Rightarrow$ Head.Next- since we are inserting to the Front this new node's next node is the current Head's Next Nodesince in this representation we assume that Head is a sentinel node.

3. NewNode.Previous $\Rightarrow$ Head – The new node'sprevious is now pointing back to Head.

4. Head.Next.Previous $\Rightarrow$ NewNode- The Head.Next is the original List's First Node and it'sPrev should now point to the NewNode, the new First Node.

5.Head.Next $\Rightarrow$ NewNode –Head's Next should now point to the new First Node that is NewNode.

Figure 4.19 Steps for Inserting at the Front of a Doubly Linked List

4.8.4.2 Insertion after the given node in DLL

Let us assume we call the function:

Insertafter(30, 20, List)

As is the case with insertion into a list represented as a linked list we need to iteratively search for the node using a search pointer (P) similar to the procedure shown in Figure 4.7. Initially $P \Rightarrow \text{Head}$ & we search until $P.\text{data} = 20$

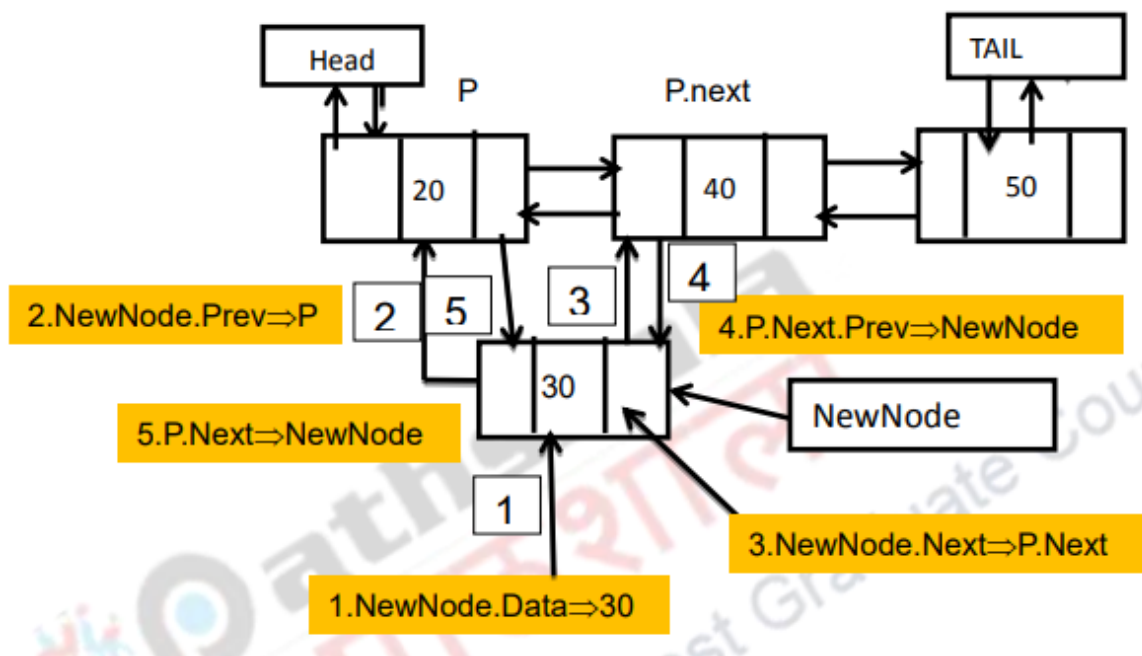


Figure 4.20 Steps for Inserting after a node of a Doubly Linked List

The insertion after a given node of a doubly linked list with sentinels is shown in Figure 4.20 and the steps are described in Figure 4.21.

Steps –Inserting After a Node of a Doubly Linked List

Freenode(NewNode) – Getting a free node in which to insert the data. The node consists of three components **Prev**(pointing to previous Node in the list), **Data** (containing data) and **Next** (pointing to next Node in the list)

1. **NewNode.Data** $\Rightarrow$ 30 – set new node's data component to 10

2. **NewNode.Prev** $\Rightarrow$ P -since we are inserting after the node P this new node's prev points to P.

3. **NewNode.Next** $\Rightarrow$ P.Next – The node after the new node should now be the node after P in the original list. Hence the Next of new node points to P.Next

4. **P.Next.Prev** $\Rightarrow$ NewNode – Now the prev of P.next which was originally pointing to P must now point to NewNode

5. **P.Next** $\Rightarrow$ NewNode –P's Next should now point to NewNode so that it is inserted after P.

Figure 4.21 Steps for Inserting After a node of a Doubly Linked List

4.8.4.3 Insertion at the end of DLL

Insertion at the end of a list is achieved without the iterative search for the end of the list if we have the sentinel Tail. The procedure is shown in Figure 4.22 and Figure 4.23 respectively.

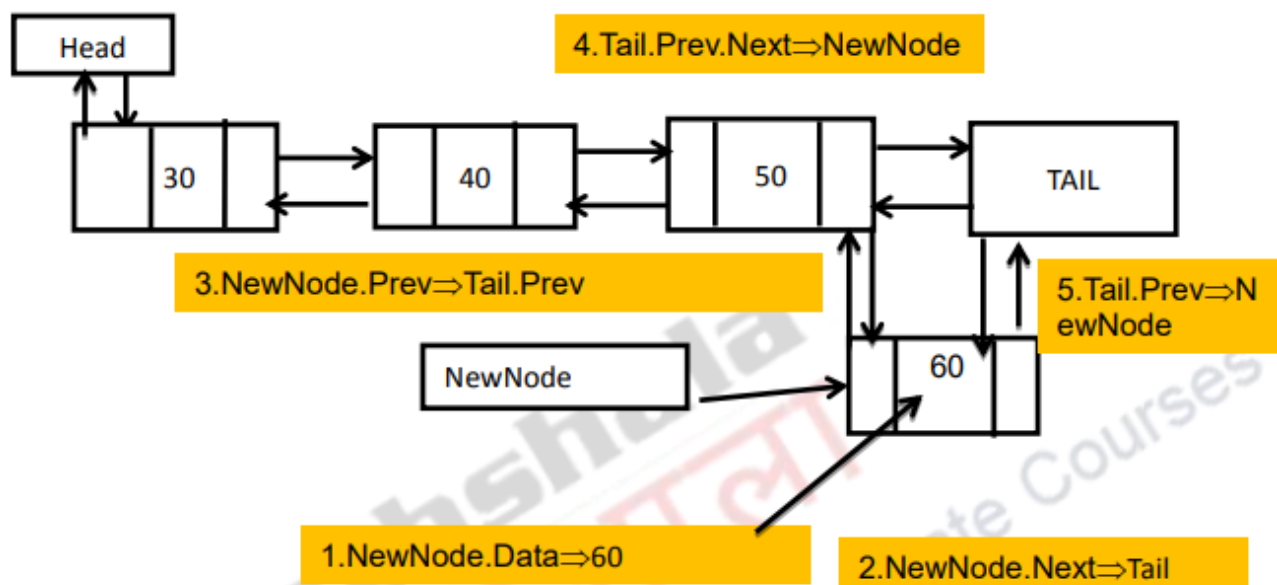


Figure 4.22 Inserting at the end of a Doubly Linked List

Steps –Inserting at the End of a Doubly Linked List

FreeNode(NewNode) – Getting a free node in which to insert the data. The node consists of three components **Prev**(pointing to previous Node in the list), **Data** (containing data) and **Next** (pointing to next Node in the list)

1. **NewNode.Data** $\Rightarrow$ 60 – set new node's data component to 60

2. **NewNode.Next** $\Rightarrow$ **Tail** – Newnode is the last node in the list and Tail is made to point to it.

3. **NewNode.Prev** $\Rightarrow$ **Tail.Prev** – The Prev of the new node points to the Prev of the Tail of the original List

4. **Tail.Prev.Next** $\Rightarrow$ **NewNode** - The Next of the Tail.Prev should now point to NewNode

5. **Tail.Prev** $\Rightarrow$ **NewNode** – Finally we update Tail.Prev to point to NewNode

Figure 4.23 Steps for Inserting at the end of a Doubly Linked List

4.8.5 Deletion from Doubly Linked List

Now we want to delete a node P from the doubly linked list and return the data value stored at P. The procedure and the steps for this deletion is given in Figure 4.24 and Figure 4.25 respectively. Here we are remove unnecessary links of P by setting it to null before removing it.

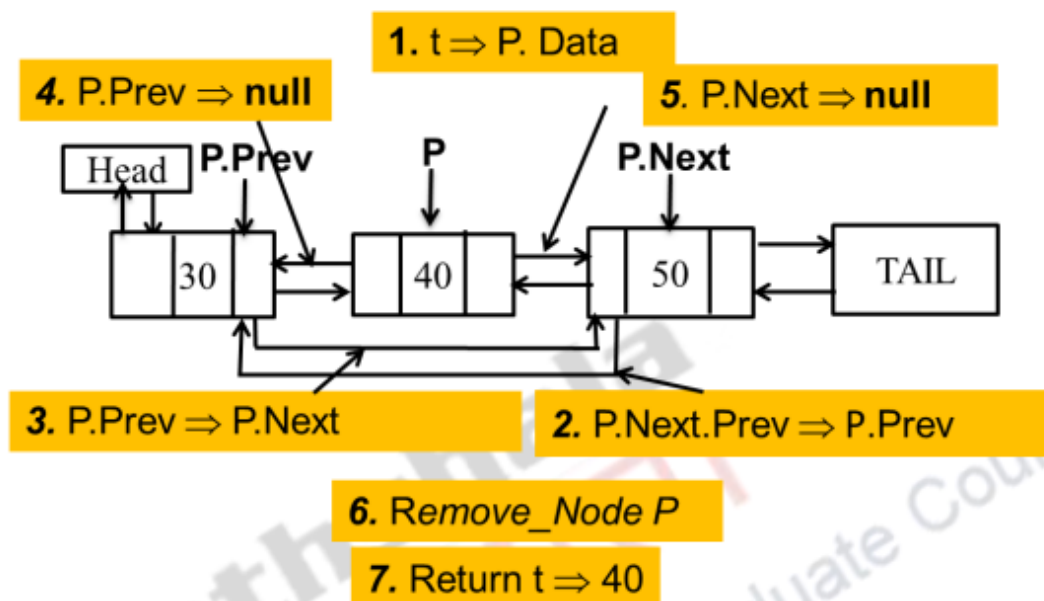


Figure 4.24 Steps to Delete a Node from a Doubly Linked List

Steps –Deleting a Node from a Doubly Linked List

1. **t $\Rightarrow$ P. Data**– Here we store the data value of P (the node we want to delete) in a variable t.
2. **P.Next.Prev $\Rightarrow$ P.Prev**– The Prev of the node after P is now made to point to the node previous to P bypassing P
3. **P.Prev.Next $\Rightarrow$ P.Next**– The Next of the node before P is now made to point to the node next to P bypassing P
4. **Remove_Node P**– Now the Node P can be removed
5. **Return t $\Rightarrow$ 40**– Now the data value of P stored in t is returned

Figure 4.25 Steps to Delete a Node from a Doubly Linked List

4.9 Performance

In the implementation of the List ADT by means of a doubly linked list the space used by a list with n elements is $O(n)$ and the space used by each position of the list is $O(1)$. All the operations of the List ADT run in $O(1)$ time.

4.10 Advantages and Disadvantages of Linked List Implementation

Advantages

Some of the advantages of linked list implementation include access to any item is possible as long as an external link to first item maintained. Insertion of a new item into the list is possible without any shifting. Similarly deletion from the list does not involve any shifting. It is possible to expand/contract the list as needed.

Disadvantages

One of the disadvantages of linked lists is the overhead due to links. Since it is used only internally for the implementation, it is pure overhead. The list adds nodes and removes nodes as and when needed so there is a need to have a facility to provide nodes and remove nodes dynamically. There is no longer direct access to each element of the list whereas many sorting algorithms for example binary search need

direct access. Access of nth item now less efficient since we must go through first element, and then second, and then third, etc.

4.11 Polynomial ADT

In this module we will also discuss the Polynomial ADT. Recall that we have studied about polynomials in mathematics. An example of a single variable polynomial having four terms is $4x^6 + 10x^4 - 5x + 3$. The order of this polynomial as determined by the highest exponent is 6. Why call it an Abstract Data Type (ADT)? Now let us consider the function $f(x)$ written as a polynomial given below that is a single variable polynomial that can be generalized as:

$$f(x) = \sum_{i=0}^n a_i x^i$$

⋮

This sum can be expanded to:

$$a_n x^n + a_{(n-1)} x^{(n-1)} + \dots + a_1 x^1 + a_0$$

Notice the two visible data sets namely: (C and E), where C is the coefficient object (Real Number) and E is the exponent object (Integer Number).

By definition a data type consists of a set of values and a set of allowable operations on those values. Associated with this polynomial the various operations can be add & subtract, multiply, differentiate, integrate, etc...

4.12 Analysis of List Implementations

In both array and linked implementations, many operations are similar in efficiency. Most operations are of the order of $O(1)$, except when shifting or searching is required in which case they are of the order of $O(n)$. In particular

situations, the frequency of the need for particular operations depending on the application may guide the use of one approach over another.

Summary

- Discussed the implementation of List ADT using Linked list
- Explained the different linked list implementations and how the different operations are implemented
- Outlined the Polynomial ADT

you can view video on **Linked List Implementation of List ADT**



Web Links
www2.latech.edu/~box/ds/chap5.ppt
www.cs.ust.hk/~huamin/COMP171/list.ppt
cs.txstate.edu/~rp44/cs3358_152/Lectures/list_revised.ppt
www.cs.binghamton.edu/~steflik/cs240/CS240PPT/Chapter06.ppt
www.comp.dit.ie/jgilligan/DT205/linked-listjg.ppt
www.cc.gatech.edu/~bleahy/cs1311/cs1311lecture09wdl.ppt
www.kau.edu.sa/Files/0005177/files/33765_Doubly-linked%20lists.ppt
Supporting & Reference Materials
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014